Rapport de travail
E3, E4, E5

Rédigé par Brahima
10 novembre 2026

Dépôt GitHub :
https://github.com/brahimadev-maker/Vigicrue-E3-5-Brahima

Sommaire

1. Organisation des fichiers **3**

2. E3 · Jeux de données Vigicrues **3**

2.1 Source des données 3

2.2 Format figé d'un scénario (v1.0) 4

2.3 Outil preparer_scenarios.py 4

2.4 Nettoyage et vérification de l'invalidité 7

2.5 Scénarios produits 8

2.6 Limites 8

3. E4 · Interface avec le rejeu **8**

4. E5 · Modes d'émission **9**

4.1 Principe 9

4.2 Mode naïf 9

4.3 Mode fréquence fixe 10

4.4 Mode S4 hybride 10

4.5 Gestion des mesures invalides 11

4.6 Intégration dans l'émulateur 11

4.7 Compilation et banc d'essai 11

4.8 Résultats 12

5. Bilan et suites **12**

Annexe A · emulateur/include/modes.h **13**

Annexe B · emulateur/src/modes.cpp **14**

Annexe C · emulateur/tests/test_modes.cpp **16**

Annexe D · E3/preparer_scenarios.py **17**

1. Organisation des fichiers

L'ensemble du travail (scénarios, outil de préparation et code des modes) est disponible sur le dépôt GitHub : https://github.com/brahimadev-maker/Vigicrue-E3-5-Brahima

Le livrable est également fourni dans l'archive `livrable_E3_E5.zip`. Son arborescence est la suivante :

```
livrable_E3_E5/
├── E3/                                  Jeux de données Vigicrues
│   ├── README.md                        Format documenté (version courte)
│   ├── preparer_scenarios.py            Outil : télécharger, nettoyer, valider
│   ├── scenario_00_SIMULE.csv           Crue simulée, sans trou (test)
│   ├── scenario_00b_SIMULE_trous.csv    Crue simulée avec trous (test)
│   ├── scenario_01_<station>.csv        Scénarios réels (5 fichiers)
│   ├── ...
│   └── cache/                           Créé par l'outil : données téléchargées
└── E5/                                  Modes d'émission
    ├── MODES.md                         Description des modes
    └── emulateur/
        ├── include/
        │   ├── mesure.h                 Structure Mesure (E4)
        │   └── modes.h                  Déclarations et réglages des modes
        ├── src/
        │   └── modes.cpp                Les trois versions de doit_emettre
        └── tests/
            └── test_modes.cpp           Banc d'essai autonome sur un CSV
```

Le fichier `mesure.h` définit la structure `Mesure` produite par le rejeu (E4) et utilisée par les modes (E5). Il est défini une seule fois et partagé par les deux parties.

2. E3 · Jeux de données Vigicrues

2.1 Source des données

Les hauteurs d'eau proviennent du réseau Vigicrues. Elles sont récupérées par l'API publique **Hub'Eau Hydrométrie, version 2**, à l'adresse `https://hubeau.eaufrance.fr/api/v2/hydrometrie`. Deux points d'accès sont utilisés :

- `/observations_tr` : mesures en temps réel d'une station (grandeur `H` pour la hauteur, en millimètres) ;
- `/referentiel/stations` : liste des stations d'un département.

Hub'Eau ne conserve ces mesures que pendant environ **30 jours**. Les scénarios doivent donc être téléchargés pendant ce délai ; une fois enregistrés en CSV, ils sont conservés définitivement. La version 1 de l'API n'est plus accessible (réponse « 403 Forbidden »).

2.2 Format figé d'un scénario (v1.0)

Élément	Valeur retenue
Type de fichier	CSV, encodage UTF-8, fins de ligne LF
Séparateur	point-virgule (;)
En-tête	une seule ligne : `t_s;date_utc;hauteur_mm`
`t_s`	temps depuis le début du scénario, en secondes ; entier, commence à 0
`date_utc`	horodatage ISO 8601 en UTC : `AAAA-MM-JJTHH:MM:SSZ`
`hauteur_mm`	hauteur d'eau en millimètres ; entier de 0 à 65534
Mesure invalide	`hauteur_mm = 65535` (0xFFFF, soit `MESURE_INVALIDE`)
Pas de temps	5 minutes (300 s), régulier : `t_s` augmente toujours de 300
Nom du fichier	`scenario_NN_<code_station>.csv`

Exemple de début de fichier :

```
t_s;date_utc;hauteur_mm
0;2026-10-29T00:00:00Z;651
300;2026-10-29T00:05:00Z;647
600;2026-10-29T00:10:00Z;649
```

La hauteur est mesurée par rapport au **zéro de l'échelle** de chaque station, et non depuis le fond de la rivière. Deux stations ne sont donc pas comparables en valeur absolue : une hauteur de 2 m peut être normale à une station et exceptionnelle à une autre. Le pas de 5 minutes est imposé, car un pas horaire ne permettrait pas de simuler les modes d'émission fins.

2.3 Outil preparer_scenarios.py

Le fichier `E3/preparer_scenarios.py` est écrit en Python 3 et utilise la bibliothèque `requests`, à installer une seule fois avec `pip install requests`. Il regroupe quatre commandes, décrites ci-dessous. Toutes se lancent depuis le dossier `E3`, sous la forme `python preparer_scenarios.py <commande> <paramètres>`. Les commandes longues sont coupées ici pour la mise en page : chacune se tape **sur une seule ligne**. L'aide d'une commande s'affiche avec `--help`, par exemple `python preparer_scenarios.py chercher --help`.

Commande	Rôle en une phrase
`chercher`	Trouve automatiquement les stations en crue et crée les scénarios.
`hubeau`	Télécharge les données brutes d'une seule station.
`nettoyer`	Transforme un fichier brut en scénario au format figé.
`valider`	Vérifie qu'un scénario respecte le format.

Commande chercher

C’est la commande principale. Elle demande à Hub’Eau la liste de toutes les stations en service dans les départements indiqués, puis télécharge les hauteurs de chacune sur la période choisie. Elle écarte les stations qui ne mesurent pas au pas de 5 minutes ou qui ont trop peu de mesures, calcule pour les autres la **montée** (hauteur maximale moins hauteur minimale), classe les stations de la plus forte à la plus faible montée, puis crée directement les scénarios des premières, déjà nettoyés et validés. Si le serveur ne répond pas, elle réessaie jusqu’à quatre fois, puis ignore la station sans s’arrêter. Les données téléchargées sont gardées dans le dossier `cache/` : en relançant la même commande, seules les stations manquantes sont retéléchargées.

Paramètre	Signification
`départements`	un ou plusieurs numéros de départements, séparés par des espaces (2A : Corse-du-Sud, 2B : Haute-Corse, 30 : Gard, 34 : Hérault)
`--debut`	début de la période étudiée, en UTC (obligatoire)
`--fin`	fin de la période étudiée, en UTC (obligatoire)
`--creer`	nombre de scénarios à créer (5 par défaut)

Exemple :

```
python preparer_scenarios.py chercher 2A 2B 30 34 --debut 2026-11-03T00:00 --fin
        2026-11-09T00:00 --creer 5
```

Ce qui s’affiche (les valeurs varient selon la station et la période) :

```
186 stations à examiner...
  ...
  [132/186] Y350000201 Le Vieux Vistre à Rodilhan  montée 0.26 m
  ...
Stations qui ont le plus monté :
  ...
scenario_01_Y340402001.csv : ...
[OK] scenario_01_Y340402001.csv : 1729 lignes, 144.0 h
```

Le nombre entre crochets est une barre de progression : `[132/186]` signifie que la station examinée est la 132e sur les 186 trouvées dans les quatre départements. Les stations écartées ne sont pas affichées, ce qui explique que la numérotation saute parfois. L’examen complet prend plusieurs minutes, car le serveur est interrogé une fois par station. Hub’Eau ne gardant que 30 jours de mesures, la période choisie doit être récente.

Commande hubeau

Elle télécharge toutes les mesures de hauteur disponibles pour **une seule station**, soit environ les 30 derniers jours, et les enregistre telles quelles dans un fichier brut. Ce fichier n'est pas encore un scénario : il n'est ni recalé sur le pas de 5 minutes, ni vérifié. Il faut ensuite le passer dans la commande `nettoyer`. Cette commande est utile lorsque l'on choisit soi-même une station, par exemple en repérant une crue sur la carte de vigicrues.gouv.fr.

Paramètre	Signification
`station`	code de la station à 10 caractères, visible sur la fiche de la station dans Vigicrues
`--sortie`	nom du fichier créé (facultatif ; par défaut `brut_<code>.csv`)

Exemple :

```
python preparer_scenarios.py hubeau J712001001
```

Ce qui s'affiche (les valeurs varient selon la station et la période) :

```
8537 mesures brutes -> brut_J712001001.csv
```

Le fichier brut contient deux colonnes, `date_utc;hauteur_mm`, avec les mesures dans l'ordre chronologique.

Commande nettoyer

Elle transforme un fichier brut en scénario au format figé. Le fichier brut peut venir de la commande `hubeau` ou d'un export téléchargé à la main sur Hydroportail (hydro.eaufrance.fr) : l'outil repère seul le séparateur, la colonne des dates et la colonne des hauteurs. Il applique ensuite les règles de nettoyage décrites en 2.4 (grille de 5 minutes, interpolation des petits trous, valeur 65535 pour les grands), écrit le scénario, affiche un résumé, puis le valide automatiquement.

Paramètre	Signification
`brut`	fichier brut à transformer
`sortie`	nom du scénario à créer, au format `scenario_NN_<code>.csv`
`--debut`, `--fin`	facultatifs : ne garder que cette période (UTC), par exemple autour du pic de crue
`--unite`	unité des hauteurs dans le brut : `mm` (par défaut), `cm` ou `m`
`--heure-locale`	à ajouter si les dates du brut sont en heure française et non en UTC

Exemple :

```
python preparer_scenarios.py nettoyer brut_J712001001.csv
        scenario_06_J712001001.csv --debut 2026-11-05T00:00 --fin 2026-11-08T00:00
        --unite mm
```

Ce qui s'affiche (les valeurs varient selon la station et la période) :

```
N mesures lues dans brut_J712001001.csv
N lignes -> scenario_06_J712001001.csv  (interpolées : X, invalides : Y, négatives
        ramenées à 0 : Z)
[OK] scenario_06_J712001001.csv : N lignes, D h
```

Le résumé indique combien de valeurs ont été interpolées (petits trous comblés), combien sont invalides (grands trous, notées 65535) et combien de hauteurs négatives ont été ramenées à 0. Pour un export Hydroportail en mètres, il faut indiquer `--unite m`.

Commande valider

Elle vérifie qu'un ou plusieurs scénarios respectent exactement le format figé, afin qu'ils puissent être lus sans erreur par le rejeu. Elle contrôle l'en-tête, la présence de trois colonnes, le fait que `t_s` et `hauteur_mm` soient des entiers, le format des dates, le départ de `t_s` à 0, le pas régulier de 300 secondes et la plage des hauteurs.

Paramètre	Signification
`fichiers`	un ou plusieurs scénarios ; le caractère * désigne plusieurs fichiers d'un coup

Exemple :

```
python preparer_scenarios.py valider scenario_01_*.csv scenario_02_*.csv
```

Ce qui s'affiche (les valeurs varient selon la station et la période) :

```
[OK] scenario_01_Y340402001.csv : 1729 lignes, 144.0 h
[OK] scenario_02_V717000901.csv : 1729 lignes, 144.0 h
```

En cas de problème, la commande affiche `[ERREUR]` suivi de la liste des anomalies et du numéro de ligne concerné, par exemple `ligne 57 : pas de 600 s au lieu de 300`.

2.4 Nettoyage et vérification de l'invalidité

La fonction `nettoyer()` applique les règles suivantes :

- tri chronologique et suppression des doublons (la dernière valeur est conservée) ;
- recalage sur une grille régulière de 5 minutes ;
- **trou de 30 minutes ou moins** : la valeur manquante est **interpolée**, c'est-à-dire estimée en traçant une ligne droite entre la mesure précédente et la suivante (par exemple, 600 mm à 10 h 00 et 620 mm à 10 h 10 donnent 610 mm à 10 h 05) ;
- **trou de plus de 30 minutes** : la ligne est conservée avec la valeur 65535, qui signifie « mesure invalide » ; la grille reste ainsi régulière et le rejeu reçoit les trous ;

- hauteur négative (eau sous le zéro de l'échelle) : ramenée à 0, car le champ est un entier non signé.

L'outil affiche un résumé (nombre de valeurs interpolées et invalides), puis la commande `valider` contrôle l'en-tête, le nombre de colonnes, les entiers, le format des dates, le départ à 0 et le pas de 300 s.

Des valeurs manquent régulièrement dans les données réelles : coupures du réseau de transmission, pannes d'alimentation des stations, capteurs endommagés par les débris pendant les crues, opérations de maintenance, ou mesures retirées après contrôle par les hydrologues.

2.5 Scénarios produits

Fichier	Station	Mesures	Durée	Invalides
scenario_00_SIMULE.csv	crue simulée	288	24 h	0
scenario_00b_SIMULE_trous.csv	crue simulée avec trous	288	24 h	16
scenario_01_Y340402001.csv	Y340402001	1729	6 jours	0
scenario_02_V717000901.csv	V717000901	1729	6 jours	0
scenario_03_Y346400501.csv	Y346400501	1729	6 jours	0
scenario_04_Y344402001.csv	Y344402001	1729	6 jours	0
scenario_05_Y340501001.csv	Y340501001	1729	6 jours	0

Les scénarios réels couvrent l'épisode de pluies intenses de début novembre 2026 (vigilance orange en Corse, dans le Gard et dans l'Hérault). Les deux scénarios simulés servent uniquement aux tests : ils ne sont pas des séries Vigicrues. Les résultats du chapitre 4 montrent que les scénarios 01 et 05 présentent une montée très faible.

2.6 Limites

- Les valeurs aberrantes (pic absurde) ne sont pas détectées : seules les mesures manquantes deviennent invalides.
- Un capteur bloqué sur une valeur constante n'est pas signalé.
- Les indicateurs de qualité de Hub'Eau (statut « Brute », qualification) ne sont pas conservés.

3. E4 · Interface avec le rejeu

Le rejeu (E4) lit les fichiers CSV de E3 et produit, dans l'ordre et au bon rythme, une structure `Mesure` par ligne. Cette structure est définie dans le fichier `emulateur/include/mesure.h` :

```
// emulateur/include/mesure.h
#pragma once
#include <stdint.h>

// Hauteur signalant une mesure invalide (capteur muet, trou de donnée).
// Alignée sur le contrat d'interface : water_mm = 0xFFFF = invalide.
static const uint16_t MESURE_INVALIDE = 0xFFFF;

struct Mesure {
    uint32_t t_s;          // temps écoulé depuis le début du scénario, en s
    uint32_t horodatage;   // date de la mesure, en secondes epoch UTC
    uint16_t hauteur_mm;   // hauteur d'eau en mm ; MESURE_INVALIDE si non valide

    bool valide() const { return hauteur_mm != MESURE_INVALIDE; }
};
```

Champ	Correspondance avec le CSV
`t_s`	colonne `t_s`, recopiée telle quelle
`horodatage`	colonne `date_utc`, convertie par E4 en secondes depuis le 1er janvier 1970 (epoch UTC)
`hauteur_mm`	colonne `hauteur_mm` ; la valeur 65535 est transmise telle quelle

La méthode `valide()` renvoie `false` lorsque la hauteur vaut `MESURE_INVALIDE` (0xFFFF). Le traitement de ces mesures relève des modes (E5). La structure étant partagée par E4 et E5, toute modification d'un champ doit être répercutée des deux côtés.

4. E5 · Modes d'émission

4.1 Principe

Pour chaque `Mesure` produite par le rejeu, l'émulateur appelle `doit_emettre(mesure, etat)`, qui répond oui ou non. La variable `etat` (structure `EtatMode`) mémorise ce qui s'est passé auparavant : heure et hauteur du dernier envoi, état d'alerte, niveau de base, etc. Chaque mode est une version de cette fonction ; le mode est choisi au démarrage.

Fichier	Contenu
`include/modes.h`	énumération `Mode`, réglages `ParamsModes`, état `EtatMode`, déclarations
`src/modes.cpp`	`doit_emettre_naif`, `doit_emettre_fixe`, `doit_emettre_s4`, le répartiteur `doit_emettre` et les outils de démarrage
`tests/test_modes.cpp`	banc d'essai : lit un CSV E3 et affiche les décisions (ne remplace pas E4)

4.2 Mode naïf

Le boîtier émet chaque mesure valide, soit une trame toutes les cinq minutes. Ce mode est le plus réactif mais le plus coûteux en énergie ; il sert de référence.

4.3 Mode fréquence fixe

Le boîtier émet une trame toutes les `periode_fixe_s` secondes (3600 s, soit une heure, par défaut). Si l'instant prévu tombe sur une mesure invalide, la mesure valide suivante est émise. Ce mode est économe, mais une montée rapide peut n'être connue qu'avec une heure de retard.

4.4 Mode S4 hybride

S4 combine un envoi périodique lent et des envois déclenchés par des événements. Il émet si l'une des conditions suivantes est remplie :

- c'est la première mesure valide ;
- le capteur revient après une panne ;
- la hauteur a varié d'au moins `s4_delta_mm` (50 mm) depuis le dernier envoi ;
- le boîtier entre en alerte ou en sort ;
- la période est écoulée : 1 h hors alerte, 15 min en alerte.

Seuil d'alerte relatif. Comme le zéro de l'échelle diffère d'une station à l'autre, un seuil fixe en millimètres ne convient pas : une première version avec un seuil de 1500 mm restait en alerte permanente sur les stations dont le niveau normal dépasse 1,5 m. S4 mémorise donc le **niveau de base**, c'est-à-dire la plus basse hauteur observée depuis le début, et passe en alerte lorsque la hauteur atteint ce niveau plus `s4_marge_alerte_mm` (500 mm). Une hystérésis de 50 mm évite d'osciller autour du seuil. Ce seuil s'adapte automatiquement à chaque station.

Réglage (ParamsModes)	Valeur	Signification
`periode_fixe_s`	3600 s	période du mode fixe
`s4_periode_calme_s`	3600 s	envoi périodique de S4 hors alerte
`s4_periode_alerte_s`	900 s	envoi périodique de S4 en alerte
`s4_delta_mm`	50 mm	variation qui déclenche un envoi
`s4_marge_alerte_mm`	500 mm	marge au-dessus du niveau de base
`s4_hysteresis_mm`	50 mm	marge de sortie d'alerte
`s4_muet_apres_s`	1800 s	délai avant signalement d'un capteur muet

Remarque : la logique ci-dessus constitue la définition retenue pour S4 ; tous les réglages sont regroupés dans `ParamsModes` pour être ajustés.

4.5 Gestion des mesures invalides

Les modes naïf et fixe ignorent les mesures invalides (`if (!m.valide()) return false;`). S4 les ignore aussi, mais compte la durée du silence : au bout de 30 minutes, il émet une seule trame portant la valeur 0xFFFF, conformément au contrat d'interface, pour signaler au serveur que le capteur est muet. Dès qu'une mesure valide revient, il émet immédiatement.

4.6 Intégration dans l'émulateur

```
#include "modes.h"

EtatMode etat;
Mode mode;
if (!mode_depuis_texte(argv[1], mode)) { /* "naif", "fixe" ou "s4" */ }
init_etat(etat, mode);                 // mode choisi au démarrage

// pour chaque Mesure m produite par le rejeu (E4) :
if (doit_emettre(m, etat)) {
    // envoyer la trame (m.hauteur_mm = 0xFFFF : trame "capteur muet")
}
```

4.7 Compilation et banc d'essai

Sous Windows avec Visual Studio, depuis la « Developer Command Prompt », dans le dossier `E5\emulateur` :

```
cl /EHsc /nologo /std:c++14 /I include src\modes.cpp tests\test_modes.cpp
        /Fe:test_modes.exe
```

Avec g++ (MSYS2, Linux) :

```
g++ -std=c++11 -Iinclude src/modes.cpp tests/test_modes.cpp -o test_modes
```

Comme plus haut, chaque commande se tape sur une seule ligne. Exécution d'un mode sur un scénario (l'option `-v` détaille chaque émission), puis comparaison des trois modes sur tous les scénarios dans l'invite de commandes :

```
test_modes.exe s4 ..\..\E3\scenario_01_Y340402001.csv -v

for %f in (..\..\E3\scenario_0*.csv) do @(echo. & echo %~nxf & test_modes.exe naif
        %f & test_modes.exe fixe %f & test_modes.exe s4 %f)
```

Le bouton « Build active file » de VS Code ne convient pas : il ne compile que le fichier ouvert et ignore le dossier `include` ainsi que `modes.cpp`.

4.8 Résultats

Scénario	Mesures	Naïf	Fixe	S4
00 simulé	288	288 (100 %)	24 (8,3 %)	59 (20,5 %)
00b simulé avec trous	288 (16 inv.)	272 (94,4 %)	23 (8,0 %)	55 (19,1 %)
01 · Y340402001	1729	1729 (100 %)	145 (8,4 %)	145 (8,4 %)
02 · V717000901	1729	1729 (100 %)	145 (8,4 %)	350 (20,2 %)
03 · Y346400501	1729	1729 (100 %)	145 (8,4 %)	448 (25,9 %)
04 · Y344402001	1729	1729 (100 %)	145 (8,4 %)	457 (26,4 %)
05 · Y340501001	1729	1729 (100 %)	145 (8,4 %)	145 (8,4 %)

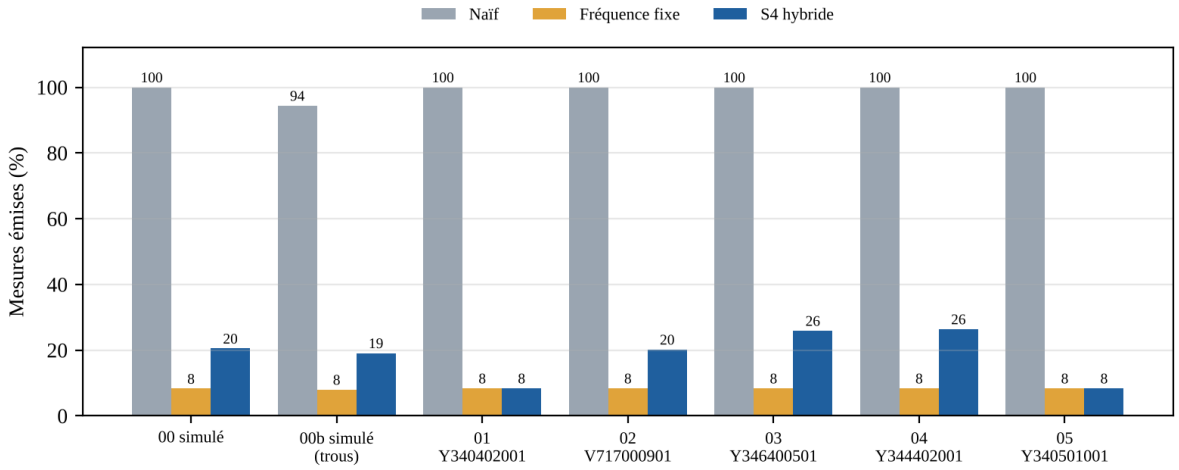


Figure 1 – Part des mesures émises selon le mode et le scénario

Le mode naïf émet la totalité des mesures valides et le mode fixe environ 8 %, conformément à leur définition. Sur les crues nettes (scénarios 02, 03 et 04), S4 émet entre 20 et 26 % des mesures : il reste économe au calme et augmente sa cadence pendant la montée et la décrue, là où l'information compte. Sur les scénarios 01 et 05, S4 se comporte exactement comme le mode fixe (145 émissions) : aucun événement n'a été déclenché, ce qui indique une montée trop faible pour constituer une crue exploitable. Ce n'est pas un défaut de S4 mais une limite de ces deux scénarios.

5. Bilan et suites

Critère de fin	État
E3 : 4 à 5 CSV existent et se lisent sans erreur	atteint (5 scénarios réels, 2 de test)
E3 : format écrit	atteint (README, chapitre 2)
E5 : trois modes implémentés	atteint (naïf, fixe, S4 hybride)
E5 : mode choisi au démarrage, décision selon sa logique	atteint (banc d'essai)

Dépôt du travail : https://github.com/brahimadev-maker/Vigicrue-E3-5-Brahima

Annexe A · emulateur/include/modes.h

```
// emulateur/include/modes.h
// E5 - Modes d'émission : décide, mesure par mesure, s'il faut émettre.
#pragma once
#include <stdint.h>
#include "mesure.h"

enum class Mode { NAIF, FIXE, S4 };

// Réglages des modes (valeurs par défaut modifiables au démarrage).
struct ParamsModes {
    // Mode fréquence fixe
    uint32_t periode_fixe_s      = 3600;  // 1 émission par heure

    // Mode S4 hybride (périodique + événementiel)
    uint32_t s4_periode_calme_s  = 3600;  // battement de cœur hors alerte
    uint32_t s4_periode_alerte_s = 900;   // cadence en alerte (15 min)
    uint16_t s4_delta_mm         = 50;    // variation qui déclenche un envoi
    uint16_t s4_marge_alerte_mm  = 500;   // alerte si hauteur >= base + marge
    uint16_t s4_hysteresis_mm    = 50;    // sortie d'alerte sous seuil - hyst.
    uint32_t s4_muet_apres_s     = 1800;  // capteur muet signalé après 30 min
};

// État mémorisé entre deux mesures.
struct EtatMode {
    Mode        mode = Mode::NAIF;
    ParamsModes p;

    bool     deja_emis         = false;
    uint32_t t_dernier_envoi   = 0;
    uint16_t h_dernier_envoi   = 0;

    // S4
    bool     en_alerte         = false;
    uint16_t niveau_base       = MESURE_INVALIDE;  // plus basse hauteur vue
    bool     a_eu_valide       = false;
    uint32_t t_derniere_valide = 0;
    bool     muet_signale      = false;

    uint32_t nb_emissions      = 0;
};

// Une version de doit_emettre par mode.
bool doit_emettre_naif(const Mesure& m, EtatMode& e);
bool doit_emettre_fixe(const Mesure& m, EtatMode& e);
bool doit_emettre_s4  (const Mesure& m, EtatMode& e);

// Point d'entrée unique : appelle la version du mode choisi au démarrage.
bool doit_emettre(const Mesure& m, EtatMode& e);

// Outils de démarrage.
void        init_etat(EtatMode& e, Mode mode, const ParamsModes& p = ParamsModes());
bool        mode_depuis_texte(const char* texte, Mode& mode);  // "naif", "fixe",
       "s4"
const char* nom_mode(Mode mode);
```

Annexe B · emulateur/src/modes.cpp

```
// emulateur/src/modes.cpp
#include "modes.h"
#include <string.h>

// Mémorise une émission (appelé quand une version décide d'émettre).
static bool emettre(const Mesure& m, EtatMode& e) {
    e.deja_emis       = true;
    e.t_dernier_envoi = m.t_s;
    e.h_dernier_envoi = m.hauteur_mm;
    e.nb_emissions++;
    return true;
}

// ---------------------------------------------------------------------------
// Mode NAÏF : émet chaque mesure valide.
bool doit_emettre_naif(const Mesure& m, EtatMode& e) {
    if (!m.valide()) return false;
    return emettre(m, e);
}

// ---------------------------------------------------------------------------
// Mode FRÉQUENCE FIXE : une émission toutes les periode_fixe_s secondes.
// Si l'instant prévu tombe sur une mesure invalide, on émet la valide suivante.
bool doit_emettre_fixe(const Mesure& m, EtatMode& e) {
    if (!m.valide()) return false;
    if (!e.deja_emis || m.t_s - e.t_dernier_envoi >= e.p.periode_fixe_s)
        return emettre(m, e);
    return false;
}

// ---------------------------------------------------------------------------
// Mode S4 HYBRIDE : périodique lent + événements.
// Seuil d'alerte RELATIF : niveau de base (plus basse hauteur vue depuis le
// début) + s4_marge_alerte_mm. S'adapte ainsi automatiquement à chaque station.
// Émet si : 1re mesure, retour du capteur, entrée/sortie d'alerte,
// variation >= delta depuis le dernier envoi, ou période écoulée
// (courte en alerte, longue sinon). Signale une fois un capteur muet
// en émettant une mesure invalide (0xFFFF, cf. contrat d'interface).
bool doit_emettre_s4(const Mesure& m, EtatMode& e) {
    const ParamsModes& p = e.p;

    if (!m.valide()) {
        if (e.a_eu_valide && !e.muet_signale &&
            m.t_s - e.t_derniere_valide >= p.s4_muet_apres_s) {
            e.muet_signale    = true;
            e.t_dernier_envoi = m.t_s;
            e.nb_emissions++;
            return true;                  // trame "capteur muet"
        }
        return false;
    }

    bool retour_capteur = e.muet_signale;
    e.muet_signale      = false;
    e.a_eu_valide       = true;
    e.t_derniere_valide = m.t_s;

    // Niveau de base : la plus basse hauteur vue jusqu'ici
    if (m.hauteur_mm < e.niveau_base)
        e.niveau_base = m.hauteur_mm;
```

```
    uint32_t seuil = (uint32_t)e.niveau_base + p.s4_marge_alerte_mm;

    // Alerte avec hystérésis (évite d'osciller autour du seuil)
    bool alerte_avant = e.en_alerte;
    if (!e.en_alerte && m.hauteur_mm >= seuil)
        e.en_alerte = true;
    else if (e.en_alerte &&
             (uint32_t)m.hauteur_mm + p.s4_hysteresis_mm < seuil)
        e.en_alerte = false;

    int32_t variation = (int32_t)m.hauteur_mm - (int32_t)e.h_dernier_envoi;
    if (variation < 0) variation = -variation;
    uint32_t periode = e.en_alerte ? p.s4_periode_alerte_s : p.s4_periode_calme_s;

    if (!e.deja_emis
        || retour_capteur
        || e.en_alerte != alerte_avant
        || variation >= (int32_t)p.s4_delta_mm
        || m.t_s - e.t_dernier_envoi >= periode)
        return emettre(m, e);

    return false;
}

// ---------------------------------------------------------------------------
bool doit_emettre(const Mesure& m, EtatMode& e) {
    switch (e.mode) {
        case Mode::NAIF: return doit_emettre_naif(m, e);
        case Mode::FIXE: return doit_emettre_fixe(m, e);
        case Mode::S4:   return doit_emettre_s4(m, e);
    }
    return false;
}

void init_etat(EtatMode& e, Mode mode, const ParamsModes& p) {
    e = EtatMode();
    e.mode = mode;
    e.p = p;
}

bool mode_depuis_texte(const char* t, Mode& mode) {
    if (!t) return false;
    if (strcmp(t, "naif") == 0) { mode = Mode::NAIF; return true; }
    if (strcmp(t, "fixe") == 0) { mode = Mode::FIXE; return true; }
    if (strcmp(t, "s4")   == 0) { mode = Mode::S4;   return true; }
    return false;
}

const char* nom_mode(Mode mode) {
    switch (mode) {
        case Mode::NAIF: return "naif";
        case Mode::FIXE: return "fixe";
        case Mode::S4:   return "s4";
    }
    return "?";
}
```

Annexe C · emulateur/tests/test_modes.cpp

```
// emulateur/tests/test_modes.cpp
// Banc d'essai autonome de E5 : lit un scénario CSV (format E3) et affiche
// les décisions d'émission. Ne remplace PAS le rejeu E4, sert juste à tester.
//
// Compilation : g++ -std=c++11 -Iinclude src/modes.cpp tests/test_modes.cpp -o
        test_modes
// Usage       : ./test_modes <naif|fixe|s4> scenario.csv [-v]
#include <stdio.h>
#include <string.h>
#include "modes.h"

int main(int argc, char** argv) {
    Mode mode;
    if (argc < 3 || !mode_depuis_texte(argv[1], mode)) {
        fprintf(stderr, "Usage : %s <naif|fixe|s4> scenario.csv [-v]\n", argv[0]);
        return 1;
    }
    bool verbeux = (argc > 3 && strcmp(argv[3], "-v") == 0);

    FILE* f = fopen(argv[2], "r");
    if (!f) { perror(argv[2]); return 1; }

    EtatMode etat;
    init_etat(etat, mode);              // mode choisi au démarrage

    char ligne[128];
    if (!fgets(ligne, sizeof ligne, f)) { fclose(f); return 1; }  // en-tête
    unsigned nb = 0, nb_invalides = 0;
    while (fgets(ligne, sizeof ligne, f)) {
        char date[32];
        unsigned long t, h;
        if (sscanf(ligne, "%lu;%31[^;];%lu", &t, date, &h) != 3) continue;
        Mesure m;
        m.t_s        = (uint32_t)t;
        m.horodatage = 0;               // fourni par le rejeu E4 dans l'émulateur
        m.hauteur_mm = (uint16_t)h;
        nb++;
        if (!m.valide()) nb_invalides++;

        if (doit_emettre(m, etat) && verbeux) {
            if (m.valide()) printf("EMET  t=%6lus  %s  %5lu mm\n", t, date, h);
            else            printf("EMET  t=%6lus  %s  CAPTEUR MUET\n", t, date);
        }
    }
    fclose(f);

    printf("Mode %-4s : %u mesures (%u invalides) -> %u emissions (%.1f %%)\n",
           nom_mode(mode), nb, nb_invalides, (unsigned)etat.nb_emissions,
           nb ? 100.0 * etat.nb_emissions / nb : 0.0);
    return 0;
}
```

Annexe D · E3/preparer_scenarios.py

```
#!/usr/bin/env python3
"""
E3 - Préparation des scénarios Vigicrues au format figé :
    t_s;date_utc;hauteur_mm   (séparateur ;, pas de 5 min, UTF-8)

Quatre commandes :
  chercher  trouve les stations qui ont connu une crue récente
            (30 derniers jours) et crée directement les scénarios
  hubeau    télécharge les ~30 derniers jours d'une station (Hub'Eau v2)
  nettoyer  transforme un fichier brut (Hub'Eau ou export Hydroportail)
            en scénario propre au pas de 5 min
  valider   vérifie qu'un ou plusieurs scénarios respectent le format

Exemples :
  python preparer_scenarios.py chercher 2A 2B 30 34 \
         --debut 2026-11-03T00:00 --fin 2026-11-09T00:00 --creer 5
  python preparer_scenarios.py hubeau J712001001
  python preparer_scenarios.py nettoyer brut.csv scenario_01_J712001001.csv \
         --debut 2026-01-10T00:00 --fin 2026-01-14T00:00 --unite mm
  python preparer_scenarios.py valider scenario_*.csv
"""
import argparse, bisect, csv, glob, re, sys
from datetime import datetime, timedelta, timezone

PAS_S = 300                # pas de temps figé : 5 min
TROU_MAX_INTERP_S = 1800   # trou <= 30 min : interpolation linéaire
INVALIDE = 65535           # = MESURE_INVALIDE (0xFFFF) de mesure.h
UTC = timezone.utc
FORMATS = ["%Y-%m-%dT%H:%M:%S", "%Y-%m-%dT%H:%M", "%Y-%m-%d %H:%M:%S",
           "%Y-%m-%d %H:%M", "%d/%m/%Y %H:%M:%S", "%d/%m/%Y %H:%M"]


# ---------------------------------------------------------------- lecture
def lire_date(texte, heure_locale=False):
    s = re.sub(r"(Z|[+-]\d\d:?\d\d)$", "", texte.strip().strip('"'))
    for f in FORMATS:
        try:
            d = datetime.strptime(s, f)
            break
        except ValueError:
            continue
    else:
        return None
    if heure_locale:
        from zoneinfo import ZoneInfo
        return d.replace(tzinfo=ZoneInfo("Europe/Paris")).astimezone(UTC)
    return d.replace(tzinfo=UTC)


def lire_nombre(texte):
    try:
        return float(texte.strip().strip('"').replace(",", "."))
    except ValueError:
        return None


def lire_brut(chemin, unite="mm", heure_locale=False):
    """Lit un fichier brut quelconque : repère la 1re colonne date et
    la 1re colonne numérique qui la suit. Renvoie [(datetime, mm), ...]."""
    facteur = {"mm": 1, "cm": 10, "m": 1000}[unite]
```

```
    with open(chemin, encoding="utf-8-sig", errors="replace") as f:
        lignes = f.read().splitlines()
    premiere = next((l for l in lignes if l.strip()), "")
    sep = max([";", ",", "\t"], key=premiere.count)
    points = []
    for champs in csv.reader(lignes, delimiter=sep):
        for i, c in enumerate(champs):
            d = lire_date(c, heure_locale)
            if d:
                v = next((lire_nombre(x) for x in champs[i + 1:]
                          if lire_nombre(x) is not None), None)
                if v is not None:
                    points.append((d, v * facteur))
                break
    return points


# ---------------------------------------------------------------- hubeau
API = "https://hubeau.eaufrance.fr/api/v2/hydrometrie"


def _get(url, params, essais=4):
    """GET avec plusieurs essais : Hub'Eau est parfois lent à répondre."""
    import requests, time
    for n in range(1, essais + 1):
        try:
            r = requests.get(url, params=params, timeout=90)
            if r.status_code in (200, 206):
                return r.json()
            if r.status_code < 500:
                return None                       # erreur définitive
        except requests.exceptions.RequestException as e:
            print(f"    (essai {n}/{essais} échoué : {type(e).__name__})")
        time.sleep(5 * n)                          # attend 5 s, 10 s, 15 s...
    return None


def telecharger(station, debut=None, fin=None):
    """Hauteurs d'une station sur Hub'Eau (30 derniers jours max).
    Renvoie [(datetime, mm), ...] triés, ou None si la station a échoué.
    Les résultats sont gardés dans le dossier cache/ : relancer la
    commande ne retélécharge pas les stations déjà faites."""
    import json, os
    os.makedirs("cache", exist_ok=True)
    cle = f"{station}_{debut:%Y%m%d%H%M}_{fin:%Y%m%d%H%M}" if debut and fin else
        None
    chemin_cache = os.path.join("cache", cle + ".json") if cle else None
    if chemin_cache and os.path.exists(chemin_cache):
        with open(chemin_cache, encoding="utf-8") as f:
            return [(lire_date(d), v) for d, v in json.load(f)]

    url = API + "/observations_tr"
    params = {"code_entite": station, "grandeur_hydro": "H", "sort": "asc",
              "size": 5000, "fields": "date_obs,resultat_obs"}
    if debut:
        params["date_debut_obs"] = debut.strftime("%Y-%m-%dT%H:%M:%SZ")
    if fin:
        params["date_fin_obs"] = fin.strftime("%Y-%m-%dT%H:%M:%SZ")
    points = []
    while url:
        page = _get(url, params)
```

```
        if page is None:
            return None                            # échec : pas de cache
```

```
        for m in page.get("data", []):
            d = lire_date(m["date_obs"])
            if d and m.get("resultat_obs") is not None:
                points.append((d, float(m["resultat_obs"])))
        url, params = page.get("next"), None

    if chemin_cache:
        with open(chemin_cache, "w", encoding="utf-8") as f:
            json.dump([(d.strftime("%Y-%m-%dT%H:%M:%SZ"), v) for d, v in points], f)
    return points


def cmd_hubeau(args):
    points = telecharger(args.station) or []
    sortie = args.sortie or f"brut_{args.station}.csv"
    with open(sortie, "w", encoding="utf-8", newline="\n") as f:
        f.write("date_utc;hauteur_mm\n")
        for d, v in points:
            f.write(f"{d.strftime('%Y-%m-%dT%H:%M:%SZ')};{v}\n")
    print(f"{len(points)} mesures brutes -> {sortie}")


def cmd_chercher(args):
    """Cherche, dans des départements, les stations au pas de 5 min qui ont
    le plus monté sur une période, et crée les scénarios correspondants."""
    import requests, time
    debut, fin = lire_date(args.debut), lire_date(args.fin)
    stations = []
    for dep in args.departements:
        page = _get(API + "/referentiel/stations",
                    {"code_departement": dep, "en_service": "true", "size": 1000})
        if page is None:
            print(f"Département {dep} : serveur injoignable")
            continue
        for s in page.get("data", []):
            stations.append((s["code_station"], s.get("libelle_station", "")))
    print(f"{len(stations)} stations à examiner...")

    resultats, echecs = [], []
    for i, (code, nom) in enumerate(stations, 1):
        pts = telecharger(code, debut, fin)
        time.sleep(0.1)
        if pts is None:
            echecs.append(code)
            print(f"  [{i}/{len(stations)}] {code} ignorée (serveur injoignable)")
            continue
        if len(pts) < 100:
            continue
        ecarts = sorted((pts[j + 1][0] - pts[j][0]).total_seconds()
                        for j in range(len(pts) - 1))
        if ecarts[len(ecarts) // 2] > 360:          # pas médian > 6 min
            continue
        vals = [v for _, v in pts]
        amplitude = max(vals) - min(vals)
        resultats.append((amplitude, code, nom, pts))
        print(f"  [{i}/{len(stations)}] {code} {nom[:40]:<40} "
              f"montée {amplitude / 1000:.2f} m")
```

```
    if echecs:
        print(f"\n{len(echecs)} station(s) ignorée(s) : relance la même commande "
              "pour les réessayer (les autres sont en cache).")
    resultats.sort(key=lambda x: x[0], reverse=True)
    print("\nStations qui ont le plus monté :")
    for amp, code, nom, _ in resultats[:10]:
```

```
        print(f"  {code}  {amp / 1000:6.2f} m  {nom}")

    for n, (amp, code, nom, pts) in enumerate(resultats[:args.creer], 1):
        grille, rap = nettoyer(pts)
        sortie = f"scenario_{n:02d}_{code}.csv"
        ecrire(grille, sortie)
        print(f"\n{sortie} : {nom}, montée {amp / 1000:.2f} m, "
              f"{rap['interpolees']} interpolées, {rap['invalides']} invalides")
        valider(sortie)


# ---------------------------------------------------------------- nettoyage
def nettoyer(points, debut=None, fin=None):
    """Trie, dédoublonne, recale sur une grille de 5 min, comble les petits
    trous, marque les grands trous INVALIDE. Renvoie (grille, rapport)."""
    uniques = {}
    for d, v in points:
        uniques[d] = v                      # doublon : la dernière valeur gagne
    pts = sorted(uniques.items())
    if debut:
        pts = [p for p in pts if p[0] >= debut]
    if fin:
        pts = [p for p in pts if p[0] <= fin]
    if len(pts) < 2:
        raise SystemExit("Pas assez de mesures dans l'intervalle choisi.")

    temps = [p[0].timestamp() for p in pts]
    valeurs = [p[1] for p in pts]
    t0 = -(-int(temps[0]) // PAS_S) * PAS_S          # arrondi au 5 min supérieur
    t_fin = int(temps[-1]) // PAS_S * PAS_S
    rapport = {"interpolees": 0, "invalides": 0, "negatives": 0}
    grille = []
    t = t0
    while t <= t_fin:
        i = bisect.bisect_left(temps, t)
        if i < len(temps) and temps[i] == t:
            v = valeurs[i]
        else:
            ta, tb = temps[i - 1], temps[i]
            if tb - ta <= TROU_MAX_INTERP_S:
                va, vb = valeurs[i - 1], valeurs[i]
                v = va + (vb - va) * (t - ta) / (tb - ta)
                rapport["interpolees"] += 1
            else:
                v = None
        if v is None:
            h = INVALIDE
            rapport["invalides"] += 1
        else:
            h = round(v)
            if h < 0:                    # uint16 : pas de hauteur négative
                h = 0
                rapport["negatives"] += 1
            h = min(h, INVALIDE - 1)
        grille.append((t - t0, datetime.fromtimestamp(t, UTC), h))
        t += PAS_S
    return grille, rapport


def ecrire(grille, chemin):
    with open(chemin, "w", encoding="utf-8", newline="\n") as f:
        f.write("t_s;date_utc;hauteur_mm\n")
        for t_s, d, h in grille:
```

```
            f.write(f"{t_s};{d.strftime('%Y-%m-%dT%H:%M:%SZ')};{h}\n")


def cmd_nettoyer(args):
    debut = lire_date(args.debut) if args.debut else None
    fin = lire_date(args.fin) if args.fin else None
    points = lire_brut(args.brut, args.unite, args.heure_locale)
    print(f"{len(points)} mesures lues dans {args.brut}")
    grille, r = nettoyer(points, debut, fin)
    ecrire(grille, args.sortie)
    print(f"{len(grille)} lignes -> {args.sortie}  "
          f"(interpolées : {r['interpolees']}, invalides : {r['invalides']}, "
          f"négatives ramenées à 0 : {r['negatives']})")
    valider(args.sortie)


# ---------------------------------------------------------------- validation
def valider(chemin):
    erreurs = []
    with open(chemin, encoding="utf-8") as f:
        lignes = f.read().split("\n")
    if lignes and lignes[-1] == "":
        lignes.pop()
    if not lignes or lignes[0] != "t_s;date_utc;hauteur_mm":
        erreurs.append("en-tête incorrect")
    t_prec = None
    for n, l in enumerate(lignes[1:], start=2):
        c = l.split(";")
        if len(c) != 3:
            erreurs.append(f"ligne {n} : 3 colonnes attendues")
            continue
        if not (c[0].isdigit() and c[2].isdigit()):
            erreurs.append(f"ligne {n} : t_s et hauteur_mm doivent être des
        entiers")
            continue
        t, h = int(c[0]), int(c[2])
        if not re.fullmatch(r"\d{4}-\d\d-\d\dT\d\d:\d\d:\d\dZ", c[1]):
            erreurs.append(f"ligne {n} : date_utc mal formée")
        if t_prec is None and t != 0:
            erreurs.append("t_s doit commencer à 0")
        if t_prec is not None and t - t_prec != PAS_S:
            erreurs.append(f"ligne {n} : pas de {t - t_prec} s au lieu de {PAS_S}")
        if h > INVALIDE:
            erreurs.append(f"ligne {n} : hauteur hors plage uint16")
        t_prec = t
        if len(erreurs) > 10:
            break
    nb = len(lignes) - 1
    if erreurs:
        print(f"[ERREUR] {chemin}")
        for e in erreurs[:10]:
            print("   -", e)
        return False
    print(f"[OK] {chemin} : {nb} lignes, {nb * PAS_S / 3600:.1f} h")
    return True


def cmd_valider(args):
    fichiers = [f for motif in args.fichiers for f in glob.glob(motif)]
    ok = all([valider(f) for f in fichiers]) if fichiers else False
    sys.exit(0 if ok else 1)
```

```
# ------------------------------------------------------------------ main
if __name__ == "__main__":
    p = argparse.ArgumentParser(description=__doc__,
                                formatter_class=argparse.RawDescriptionHelpFormatter
        )
    sp = p.add_subparsers(dest="cmd", required=True)

    a = sp.add_parser("hubeau", help="télécharger ~30 jours depuis Hub'Eau")
    a.add_argument("station")
    a.add_argument("--sortie")
    a.set_defaults(f=cmd_hubeau)

    a = sp.add_parser("chercher", help="trouver les crues récentes et créer les
        scénarios")
    a.add_argument("departements", nargs="+", help="ex. 2A 2B 30 34")
    a.add_argument("--debut", required=True, help="ex. 2026-11-03T00:00 (UTC)")
    a.add_argument("--fin", required=True, help="ex. 2026-11-09T00:00 (UTC)")
    a.add_argument("--creer", type=int, default=5, help="nombre de scénarios à
        créer")
    a.set_defaults(f=cmd_chercher)

    a = sp.add_parser("nettoyer", help="brut -> scénario au format figé")
    a.add_argument("brut")
    a.add_argument("sortie")
    a.add_argument("--debut", help="ex. 2026-01-10T00:00 (UTC)")
    a.add_argument("--fin", help="ex. 2026-01-14T00:00 (UTC)")
    a.add_argument("--unite", choices=["mm", "cm", "m"], default="mm",
                   help="unité de la hauteur dans le fichier brut")
    a.add_argument("--heure-locale", action="store_true",
                   help="les dates du brut sont en heure française")
    a.set_defaults(f=cmd_nettoyer)

    a = sp.add_parser("valider", help="vérifier un ou plusieurs scénarios")
    a.add_argument("fichiers", nargs="+")
    a.set_defaults(f=cmd_valider)

    args = p.parse_args()
    args.f(args)
```